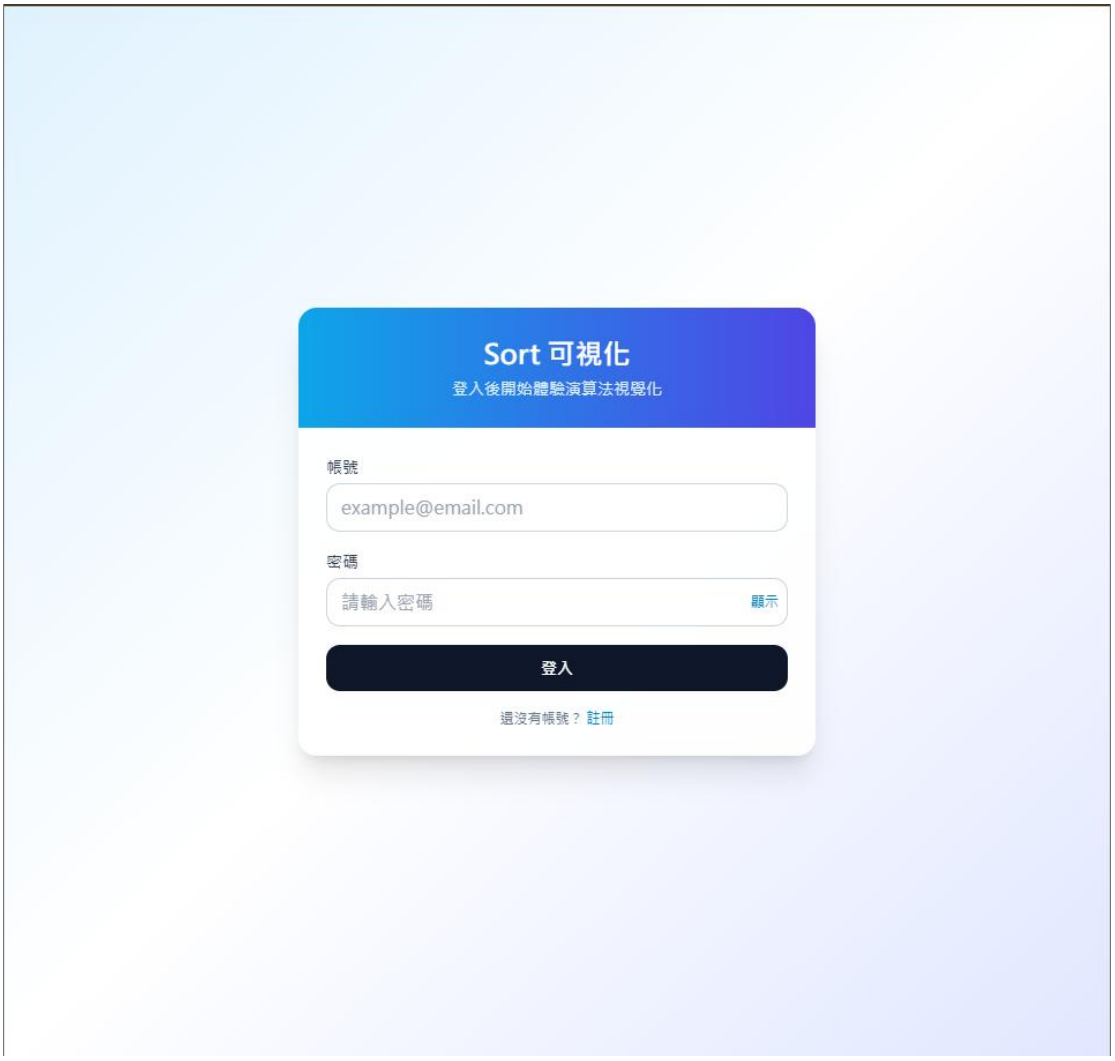
題目: 期末專題前端畫面

- 程式執行畫面:

登入畫面



登入近來畫面

SORT VISUALIZER
排序演算法視覺展示
選擇想看的排序法後，會跳出操作視窗來顯示輸入、速度控制與排序過程，方便之後繼續新增更多演算法。
Sort 01
基礎排序法
Bubble Sort
反覆比較相鄰元素，若順序錯誤就交換，較大的值會一步一步往右移動。
相鄰比較
容易理解
交換頻繁
教學常見
動畫明顯
Sort 02
基礎排序法
Selection Sort
每輪挑出目前最小值，放到最前面，能清楚看出未排序區和已排序區的差異。
尋找最小值
交換次數少
區塊清楚
教學常見
步驟直觀
Sort 03
基礎排序法
Insertion Sort
像整理手牌一樣，逐一把元素插入已排序的區間，對近乎排序好的資料很有效。
逐步插入
實作簡單
位移明顯
小資料適合
動畫清楚

隨機選擇其中一個演算法裡面的畫面

- AI 使用狀況:

  Q: 如何讓它變成漸層顏色?

  A: 使用 Tailwind CSS 語法，實作出一個由左向右（to-r）延伸的線性漸層背景，視覺上會從天藍色（sky-500）作為起點，過渡到深藍色（indigo-600）作為終點。

```
<div class="bg-gradient-to-r from-sky-500 to-indigo-600 text-white p-6 text-center">
```

Q: 如何讓區塊有浮起來的感覺?

A: 透過 Tailwind CSS 的 transition 開啟平滑轉場，並結合 hover:-translate-y-1（微幅上移）與 hover:shadow-lg（陰影加深），營造出元素向上懸浮的立體感。

```
<button class="algo-card group rounded-[2rem] border-2 border-slate-300 bg-white px-6 py-7 text-left shadow-sm transition hover:-translate-y-1 hover:border-slate-900 hover:shadow-lg" data-algo="selection">
```

Q: 如何製作模糊背景?

A: 利用 absolute inset-0 確保全方位覆蓋父容器，並結合帶有 Alpha 通道的深色背景 bg-slate-950/70 來壓低視覺干擾。最關鍵的 backdrop-blur-sm 並非模糊遮罩本身，而是對其下方的內容進行即時運算模糊，營造出一種「毛玻璃隔層」的視覺層次感。這種做法能有效引導使用者聚焦於彈出的 Modal 視窗，同時保留背景的色彩層次，避免畫面因過於死黑而顯得沈重。

```
<div class="absolute inset-0 bg-slate-950/70 backdrop-blur-sm"></div>
```

Q: 如何將使用者輸入的陣列變成長條圖?

A: 首先，系統會將使用者輸入的原始字串進行預處理，透過 split(",") 拆分並利用 map(Number) 將文字精確轉換為純數字陣列；接著進入渲染（Dynamic Rendering）階段：程式會先清空容器，並透過 Math.max 找出數值天花板作為比例基準，計算出每個數據對應的像素高度。最後，利用 JavaScript 實作 DOM 操作，動態建立帶有 Tailwind CSS 樣式的 div 元素。

```javascript
function parseInputValues(text) {
  const parts = text
    .split(",")
    .map((item) => item.trim())
    .filter(Boolean);

  if (parts.length < 2 || parts.length > 24) {
    throw new Error("請輸入 2 到 24 個整數。");
  }

  return parts.map((item) => {
    const value = Number(item);

    if (!Number.isInteger(value)) {
      throw new Error("每一個項目都必須是整數。");
    }

    return value;
  });
}
```

```javascript
function renderBars(compareIndexes = [], sortedIndexes = [], swapIndexes = []) {
  barContainer.innerHTML = "";
  const maxValue = Math.max(...values.map((value) => Math.abs(value)), 1);

  values.forEach((value, index) => {
    const wrapper = document.createElement("div");
    wrapper.className = "flex flex-1 flex-col items-center justify-end";

    const bar = document.createElement("div");
    const label = document.createElement("span");
    const isComparing = compareIndexes.includes(index);
    const isSorted = sortedIndexes.includes(index);
    const isSwapping = swapIndexes.includes(index);
    const normalizedHeight = Math.max((Math.abs(value) / maxValue) * 250, 18);

    let colorClass = "bg-sky-400";
    if (isSorted) colorClass = "bg-emerald-400";
    else if (isSwapping) colorClass = "bg-rose-500";
    else if (isComparing) colorClass = "bg-amber-400";

    bar.className = `w-full rounded-t-2xl transition-all duration-300 ${colorClass}`;
    bar.style.height = `${normalizedHeight}px`;
    bar.style.transform = isSwapping
      ? "translateY(-10px) scale(1.03)"
      : isComparing
        ? "translateY(-4px)"
        : "translateY(0)";

    label.className = `mt-2 text-[10px] font-semibold ${
      isSorted
        ? "text-emerald-300"
        : isSwapping
          ? "text-rose-300"
          : isComparing
            ? "text-amber-200"
            : "text-slate-200"
    }`;
    label.textContent = value;

    wrapper.appendChild(bar);
    wrapper.appendChild(label);
    barContainer.appendChild(wrapper);
  });
}
```

Q: 如何讓兩個柱子互換位置?

A: 當 [values[i], values[j]] 的解構賦值完成後，緊接著呼叫 renderBars() 重新渲染介面，並配合 await sleep(delay) 創造視覺停頓；這一連串「狀態更新 → 畫面重繪 → 延遲等待」的循環，讓使用者能清晰捕捉到數據變化的軌跡。這種做法捨棄了繁瑣的 DOM 位置計算，轉而透過數據同步來實現動畫。

```javascript
[values[j], values[j + 1]] = [values[j + 1], values[j]];
renderBars([], sorted, [j, j + 1]);
await sleepWithPause(delay, token);
```

Q: 怎樣調整交換的速度?

A:將 HTML 滑桿（Range Input）的數值與 JavaScript 的非同步等待時間（Sleep Duration）進行即時綁定。

當使用者操作滑桿時，系統會動態攔截 input 事件並更新全域的 delay 變數；此數值隨後被注入到 sleep 函式中，控制排序演算法在執行 DOM 更新後的停頓毫秒數。視覺上的「快慢」本質上是等待時間的縮影：滑桿數值越小，setTimeout 觸發越快，演算法的每一步躍遷就顯得越快。

```
speedRange.addEventListener("input", () => {
  delay = Number(speedRange.value);
  updateSpeedLabel();

  if (!isAnimating) {
    setStatus(`動畫速度已調整為 ${delay} ms`);
  }
});

async function sleepWithPause(ms, token) {
  let remaining = ms;

  while (remaining > 0) {
    ensureActive(token);

    while (isPaused) {
      ensureActive(token);
      await new Promise((resolve) => setTimeout(resolve, 60));
    }

    const step = Math.min(remaining, 30);
    await new Promise((resolve) => setTimeout(resolve, step));
    remaining -= step;
  }
}
```

- 心得:

我製作一個排序演算法的可視化系統，一開始我以為只是單純把排序寫出來，但實際操作之後才發現，將演算法「視覺化」比單純寫程式困難許多，也讓我對排序的理解更加深入。

在製作過程中，我首先學會了如何將使用者輸入的資料（字串）轉換成數字陣列，並透過 JavaScript 動態產生長條圖，讓每一個數字用不同高度的柱子表示。這讓我理解到資料與畫面之間的關係，也知道只要改變資料，畫面就會跟著更新。

另外，我也實作了排序動畫的效果，例如 Bubble Sort、Selection Sort 和 Insertion Sort。在這個過程中，我透過「交換陣列中的數值，再重新繪製畫面」來達到柱子交換的效果，這讓我理解到動畫不一定需要直接操作畫面元素，而是可以透過資料變化來呈現。

在速度控制方面，我利用滑桿來調整動畫延遲時間（delay），讓使用者可以自由控制排序快慢，這也讓我學會如何將介面操作與程式邏輯結合。除此之外，我也加入了暫停與繼續的功能，使整體操作更加完整。

在介面設計上，我使用了 Tailwind CSS，學會如何快速製作具有現代感的 UI，例如卡片式設計、滑鼠懸停浮起效果，以及 modal 視窗搭配背景模糊（backdrop blur）。這些設計讓整個作品看起來更像實際網站，而不只是單純的作業。

透過這次作業，我不僅加強了 JavaScript 的操作能力，也更理解了演算法的執行過程，尤其是在視覺化之後，可以更清楚看到每一步的變化。我覺得這是一個結合前端設計與程式邏輯的很好練習，也讓我對未來製作更完整的互動式網頁產生興趣。

- 參考資料:

1. 老師講義: https://github.com/nlhsueh/teach_web
2. W3School: https://www.w3schools.com/
3. ChatGPT: https://chatgpt.com/
4. CODEX: https://chatgpt.com/codex
5. GoogleAI: https://www.google.com/